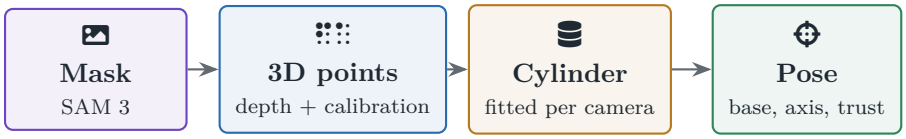
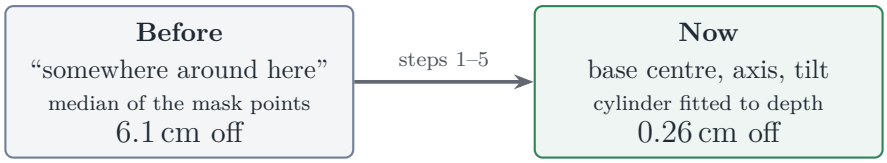
Where Is the Mug?

Object poses from the depth cameras:
what we built in steps 1–5, and why





Key idea

The robot can now say *where the bottom of the mug is* to within a few millimetres, and *which way the mug is pointing* to about a degree, using only its cameras and its own joint sensors. The first thing this is used for is a safety check: never open the gripper on a mug that is lying on its side, even when the old surface test is fooled.

Contents

1 Why the robot needs a pose **2**
1.1 The rules we kept . . . 2

2 The whole pipeline on one page **3**
2.1 Category, not model . . . 4

3 Step 1: measuring before improving **4**
3.1 The answer key was wrong by two centimetres . . . 4
3.2 Which way is “up” for a mug model . . . 4

4 Step 2: from depth to points and normals **5**

5 Step 3: fitting a cylinder **6**
5.1 The trap, and how we avoided it . . . 7

6 Removing the table, and a shadow **7**

7 Step 4: how much to trust a fit **8**
7.1 Trusted and untrusted: an asymmetric rule . . . 8
7.2 More than one camera . . . 9

8 Placing the base **9**

9 Step 5: the release check uses the axis **9**

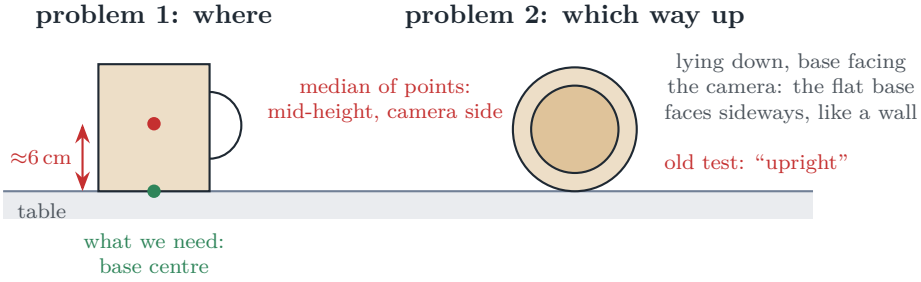
10 Results **10**

11 What is still open **11**

12 Commands **11**

1 Why the robot needs a pose

Until now, "where is the mug" was answered with one number per axis: the **median** of the 3D points inside the mug's mask. That is a fine "roughly here", but it has two problems that matter for the supervisor.



1. **Position.** The median lands about half a mug height above the base and is pulled towards the camera. On seeds 0–4 it was **6.1 cm** from the true base centre for mugs on the table. A grasp or a placement aimed there is aimed at air.
2. **Orientation.** "Safe to open the gripper" (`release_ready`) checked that the mug's visible surface faces sideways, as an upright mug's wall does. A mug lying with its base towards the camera fools it, because a flat base also faces sideways. On held-out seed 6 this produced **25 wrong "safe to release"** answers.

In plain words

Knowing *where a cup is* is not enough to pick it up or put it down. You need to know where its bottom is and whether it is standing or lying. A pose is those two things together.

1.1 The rules we kept

✔ Allowed at runtime

RGB-D images and their calibration
SAM 3 masks
the robot's own measured joints,
hand poses and grippers

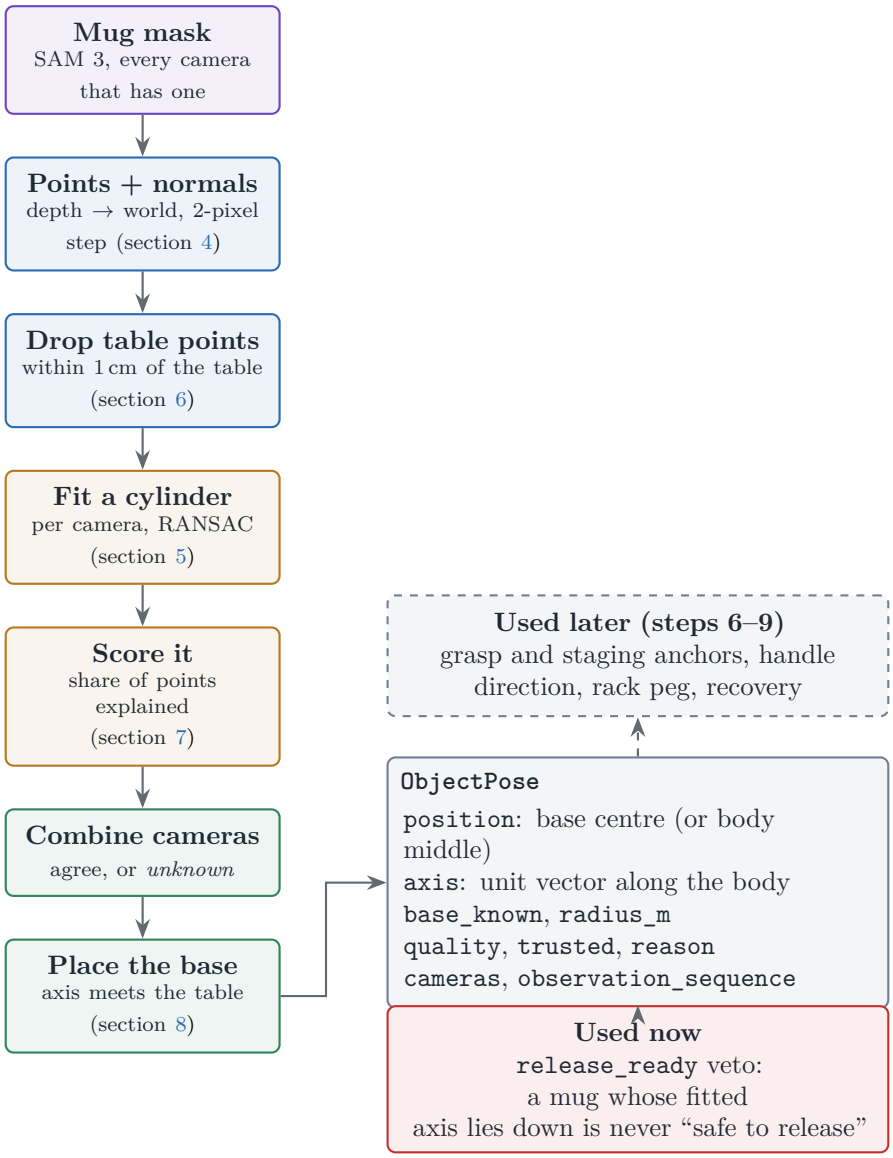
✖ Offline only

simulator object poses (labels)
the mug meshes
which mug model was loaded

⚖ How we decide

unknown beats a guess
precision before recall
tune on seeds 0–4, check on others

2 The whole pipeline on one page



Where it lives in the code

- `harness/perception/object_pose.py`: everything in the left column, plus `ObjectSpec` and `ObjectPose`.
- `harness/perception/rgbd.py`: `RGBDCamera.surface_points_and_normals`.
- `harness/perception/grounding.py`: `RGBDPerceptionProvider._mug_pose` runs it once per observation; the result is `GroundingResult.mug_pose`.
- `development/replays/perception_evaluation.py`: pose metrics against the labels.

2.1 Category, not model

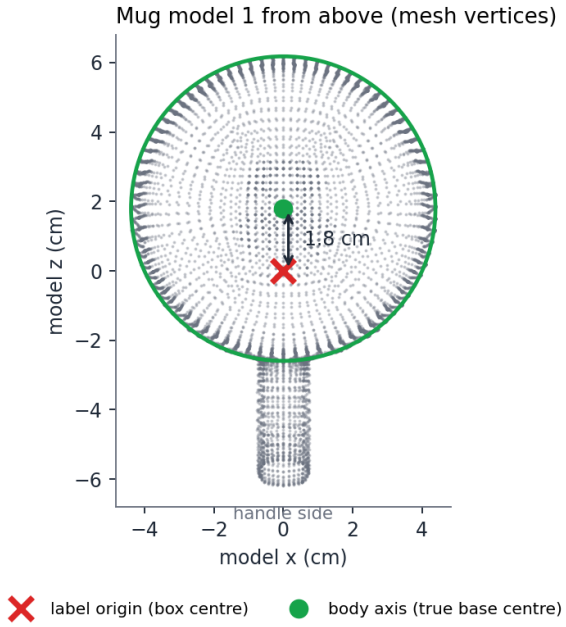
The fitter knows a mug only as an `ObjectSpec`: "a cylinder with radius 3–7 cm and height 6–14 cm that stands on the table". It never looks up which of RoboTwin's ten mug meshes was loaded. A different mug, or a bottle, needs a new spec line, not new code. This was one of the decisions in the plan: no mesh files at runtime.

3 Step 1: measuring before improving

Every change here was judged by numbers from the existing evaluator, which replays saved frames through the real runtime code and compares with the simulator's labels. We added pose metrics: axis error, base position error, upright/toppled mistakes, coverage, and **confident wrong poses** (trusted, but off by more than 20° or 3 cm). That last one is the pose version of a false "true".

3.1 The answer key was wrong by two centimetres

The first run gave a stubborn 1.9 cm horizontal error on *every* mug on the table, with almost no spread. A constant error is usually a definition problem, not a perception one.



Careful

The label's `mug.pose` origin is the centre of a box that *includes the handle*. The mug's body axis sits **1.6–2.6 cm** away from it, on the side away from the handle, depending on the model.

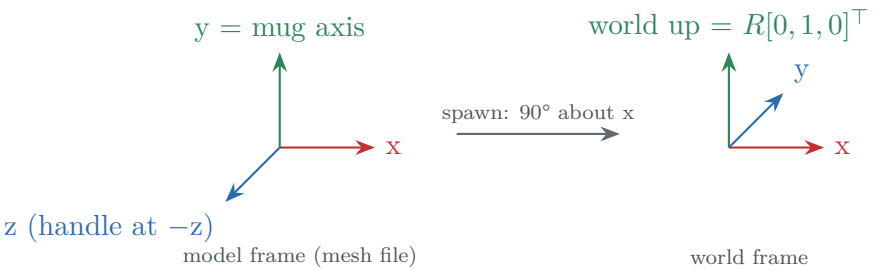
What we did (offline only):

- measured each model's offset from its mesh (`MUG_MODELS`);
- recognised the episode's model from its handle point, which is different for every model;
- moved the truth onto the body axis (`true_mug_base`);
- made new captures record `mug.model_id` directly.

Error after the fix: **0.26 cm**.

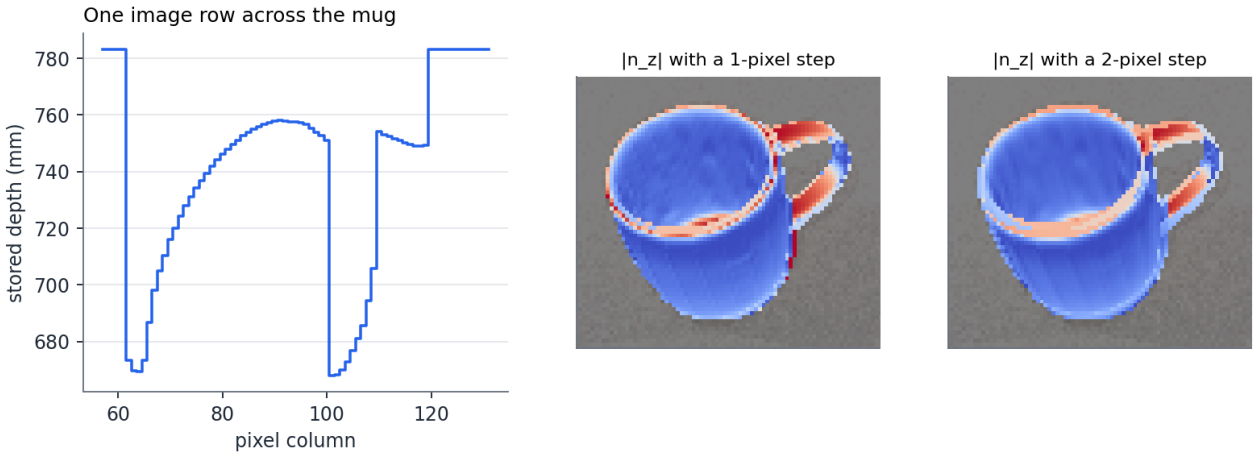
3.2 Which way is "up" for a mug model

The meshes stand along their **+y** axis. The task spawns them turned 90° about x, so +y points up in the world. The truth axis is therefore $R\,[0,1,0]^\top$, not the quaternion's z axis as the first draft of the plan said.



4 Step 2: from depth to points and normals

Each mask pixel with a depth becomes a world point (as before). We also need its **surface normal**: which way the surface faces. The normal at a pixel comes from the depth of its neighbours left/right and up/down.



- **Depth is stored in whole millimetres** (left: the staircase). Neighbours one pixel apart often differ by exactly 0 or 1 mm, which makes normals jumpy. Measuring across **two pixels** on each side averages that rounding out; the wall colour is a little more even on the right. We did not run a separate test of one step against the other.
- **A bug we hit:** pixels next to an edge are skipped when neighbouring depths jump by more than 1 cm. With the wider step, the two neighbours are 4 pixels apart, so a steep but smooth surface also "jumps". A toppled mug kept only **109 of its 1618 mask pixels**. The limit now grows with the step.

Where it lives in the code

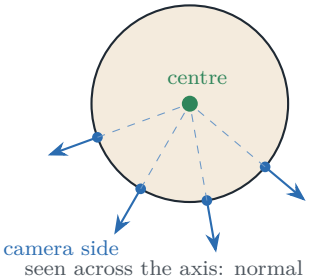
`RGBDCamera.surface_points_and_normals(mask, step_px=...)` returns world points and unit normals (turned to face the camera). The old `vertical_normal_components` now calls it, so the existing surface test is unchanged.

5 Step 3: fitting a cylinder

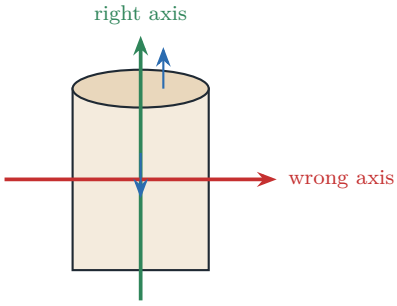
A mug's body is close to a cylinder. A cylinder has a simple property we can test point by point:

Key idea

On the curved wall, every surface normal is **perpendicular to the axis** and **points at the axis**. Two wall normals are enough to guess the axis (their cross product), and their lines meet at the centre.



seen across the axis: normal lines meet at the centre, even when only a narrow arc is visible

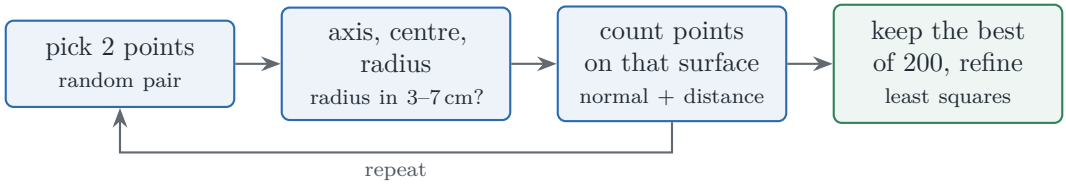


from above, the front wall (facing us) and the flat rim (facing up) are *both* perpendicular to a horizontal axis too

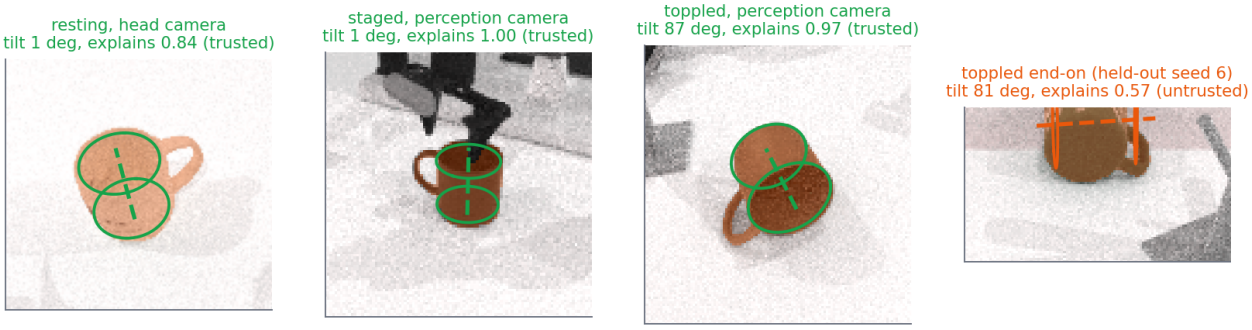
5.1 The trap, and how we avoided it

Our first version scored an axis guess only by "how many normals are perpendicular to it". From an elevated camera that let a **horizontal** axis explain an upright mug's front wall and rim (right drawing). In the first test on seeds 0–4, more than a dozen per-camera fits of upright mugs came out about 90° wrong.

The fix is to test a *whole cylinder*, not just a direction. Each random pair of points fixes the axis, the centre (where their normal lines meet) and the radius; a point only counts if its normal fits *and* it lies on that cylinder's surface (within 4 mm). Wrong axes stop adding up.



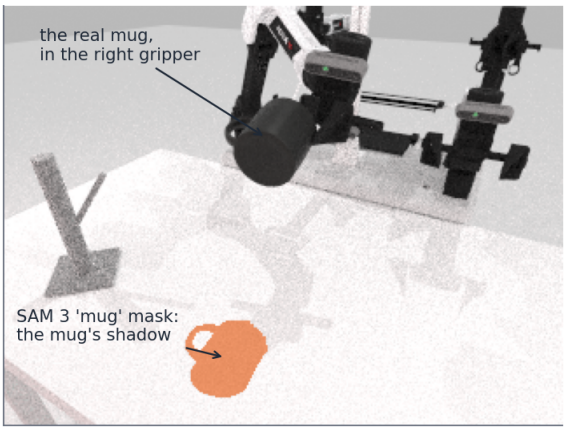
End caps. When a mug's flat base faces the camera squarely there is almost no wall to see. If most normals agree, the points are flat, and the patch is *round* with a mug-sized radius, it is an **end cap** and the axis is its normal. "Round" matters: a flat strip along a lying mug's side passed the first version of this test.



Fitted cylinders drawn back onto real frames (orange = SAM 3 mask). Right: the held-out end-on mug that fooled the old test. The fit reads it as lying down (81°), but it explains only 57% of its points, so it is not trusted (section 7).

6 Removing the table, and a shadow

Masks spill a little onto the table past the mug's edge. Points within 1 cm of the table top are dropped before fitting. This also fixed a detector mistake:



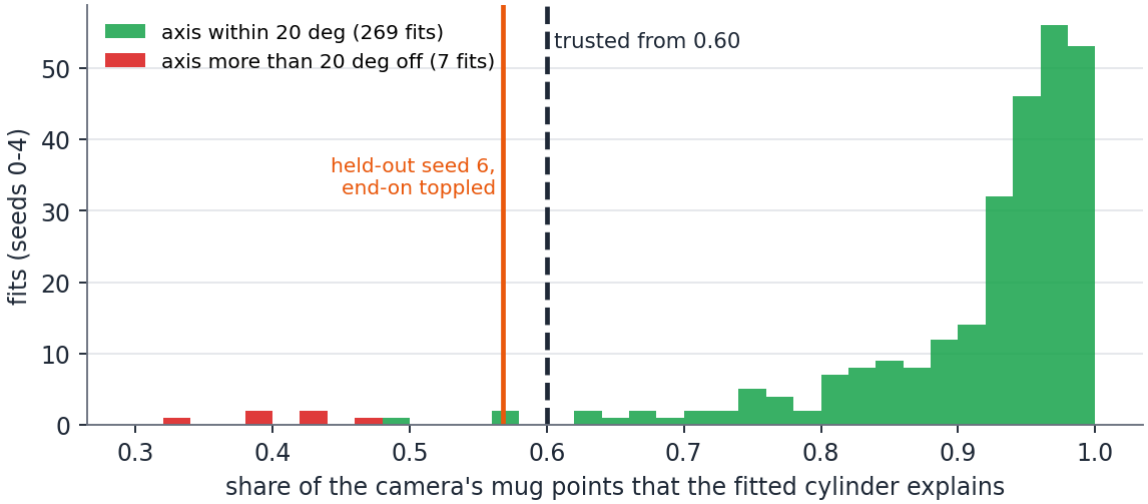
Here SAM 3 labelled the mug's **shadow** as the mug, while the real mug was in the right gripper. The shadow is flat and round, so it looked like an end cap standing upright.

Every shadow point lies on the table, so after the table points are removed nothing is left to fit, and the pose is *unknown*, which is the right answer.

The table height comes from the current frame's table mask, the same points the support check already uses.

7 Step 4: how much to trust a fit

A fit always returns *some* cylinder. We need a number that says whether to believe it. We use the share of the camera's mug points that the fitted cylinder **explains**: they lie on its wall, or on a flat end inside its radius (the base, or the floor inside the cup).

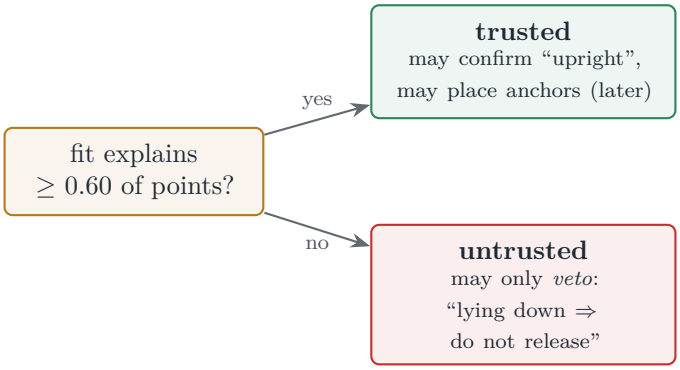


On seeds 0–4 the wrong fits (red, all mugs half hidden by a gripper) explain 47% of their points or less. Almost all right fits explain far more. The floor of **0.60** sits in that gap. This histogram uses every camera with a mug mask. In a real run the detector asks only some cameras each phase; for those fits, every wrong fit was below 0.44 and every right one above 0.70.

7.1 Trusted and untrusted: an asymmetric rule

The held-out end-on mug (orange line) was read correctly as lying down, but at 0.57 it fell just below the floor and was thrown away, so the old test let it through.

Lowering the floor until it passed would be tuning on the test set. Instead we applied the project's precision rule: **a reading that blocks an unsafe action needs less confidence than one that allows it**.

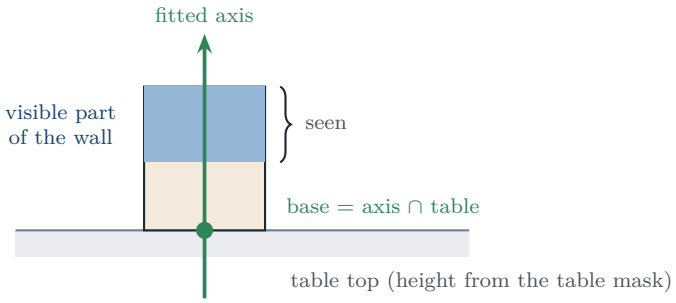


A wrong veto costs a little recall: the supervisor waits. A wrong "safe" drops the mug.

7.2 More than one camera

Each camera is fitted on its own. Trusted fits must agree (axes within 15°, axis lines within 3 cm). If they disagree the pose is *unknown* with reason `cameras_disagree`; we do not pick a favourite. The best-supported fit gives the pose.

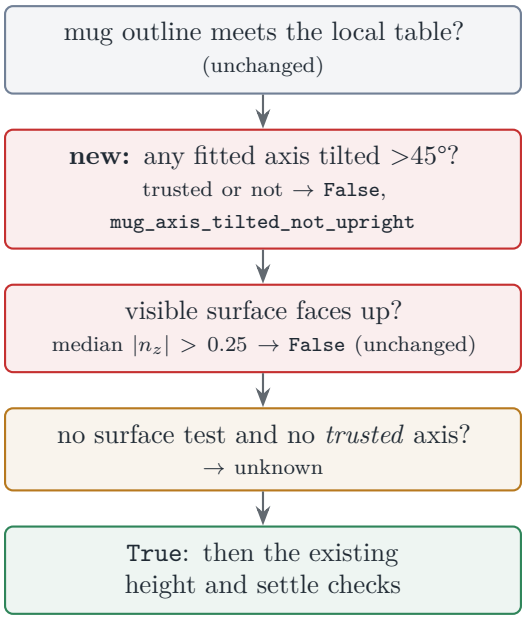
8 Placing the base



If the axis is within 45° of vertical (the same cut-off the labels use for "toppled") and the lowest visible part comes within 3 cm of the table, the mug stands on the table. Its base centre is then where the axis meets the table top, **even though the base itself is hidden**. That is what removed the 5.7 cm height error.

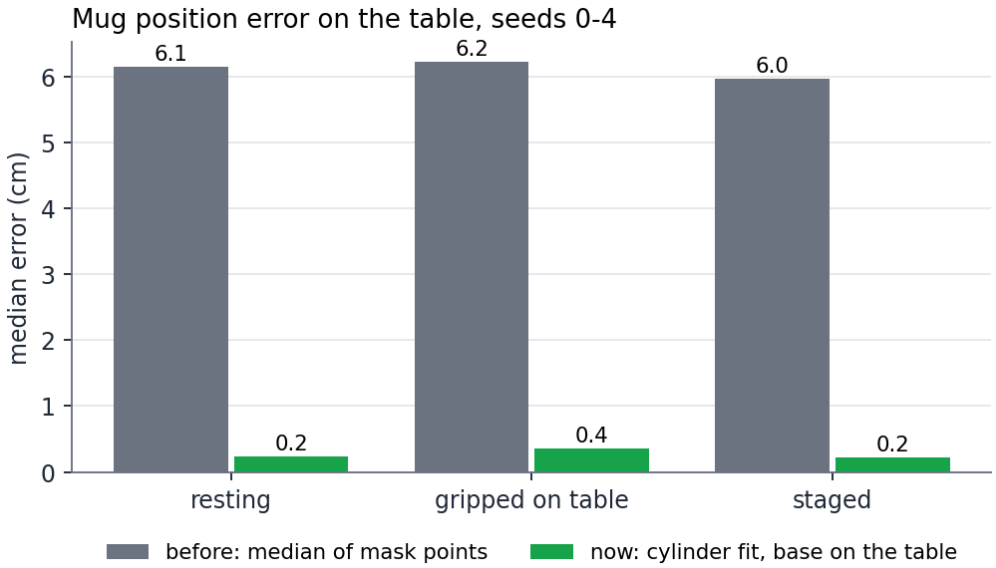
Otherwise (lying down, or lifted) `base_known` is false and `position` is the middle of the visible body. Which end of a lying mug is the base is step 6.

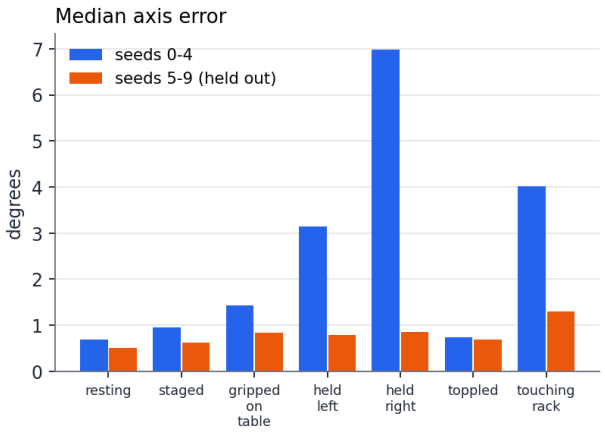
9 Step 5: the release check uses the axis

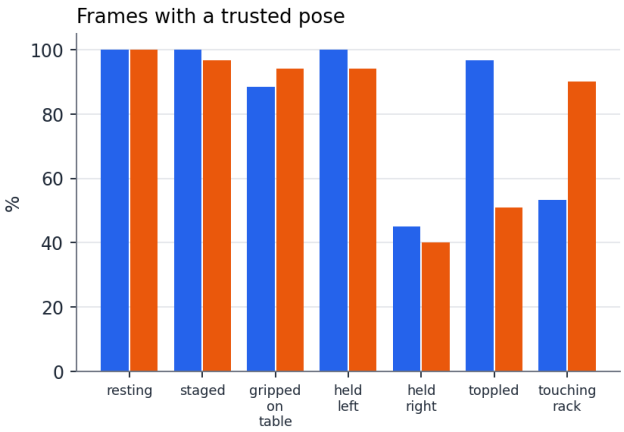


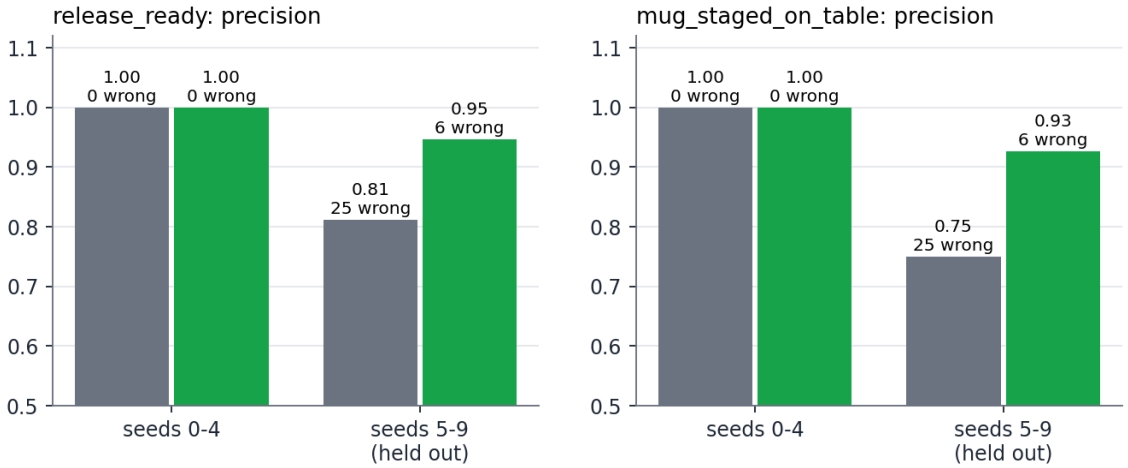
Either test can say "not upright". A trusted upright axis can also stand in for the surface test when the mask was too small for it (the old "sliver" case).

10 Results


Mug position error on the table, seeds 0-4
median error (cm)
6.1
0.2
6.2
0.4
6.0
0.2
resting
gripped on table
staged
before: median of mask points
now: cylinder fit, base on the table


Median axis error
seeds 0-4
seeds 5-9 (held out)
degrees
resting
staged
gripped on table
held left
held right
toppled
touching rack


Frames with a trusted pose
%
resting
staged
gripped on table
held left
held right
toppled
touching rack


release_ready: precision
1.00
0 wrong
1.00
0 wrong
0.81
25 wrong
0.95
6 wrong
seeds 0-4
seeds 5-9 (held out)
mug_staged_on_table: precision
1.00
0 wrong
1.00
0 wrong
0.75
25 wrong
0.93
6 wrong
seeds 0-4
seeds 5-9 (held out)
surface direction only
plus the fitted axis

Run	Posed	Axis error	Base error	Up/toppled mix-ups	Confident wrong
seeds 0–4, with perception camera	159/216	1.1°	2.6 mm	0	0
seeds 0–4, without it	98/216	1.5°	2.5 mm	0	0
seeds 5–9, with perception camera	188/267	0.7°	2.8 mm	0	0
seeds 5–9, without it	157/267	0.7°	2.7 mm	0	0

- **No predicate lost anything**: every precision and recall on seeds 0–4 is identical to before.
- **Held-out "safe to release" false trues: 25 → 6**; precision 0.81 → 0.95.
- "Posed" counts trusted poses only. Most frames without one are mugs off the table or hidden by a gripper; the pose then says *unknown* with a reason.

11 What is still open

Careful

- **Seeds 5–9 are no longer a clean test.** We looked at seed 6 while designing the trusted/untrusted rule. The 0.95 must be confirmed on **fresh seeds 10–14**.
- **6 wrong "safe to release" remain**, all seed 6's end-on mug: 4 frames where the untrusted fit does not read as lying down, 2 where no cylinder fitted at all.
- **Gripper points are not removed.** The measured hand pose sits about 0.2 m behind the fingers, so a box around it would delete the mug. Held mugs rely on the trust score instead; only 40–45% of `held_right` frames get a trusted pose.



12 Commands

Run from the repository root. Nothing here needs a GPU: the SAM 3 answers are cached.

Tests (CPU, seconds):

```
conda run -n robotwin python -m pytest -q tests/test_object_pose.py \
tests/test_perception_grounding.py tests/test_perception_scoring.py
```

Score poses and predicates (under a minute per five seeds):

```
conda run -n robotwin python -m development.replays.perception_evaluation \
--frames data/perception/frames_cam/seed_0 \
...  (one --frames per seed, up to seed_4) \
--output data/perception/reports/pose_dev_with_camera
```

Add `--ignore-camera perception_camera` to score without the new camera. The report's `summary.md` has a "Mug pose by state" table; `frames.jsonl` has the per-frame pose errors and reasons.

Rebuild this guide:

```
conda run -n robotwin env PYTHONPATH=$PWD python \
docs/Collaboration/Olusola/guides/pose_estimation/make_figures.py
cd docs/Collaboration/Olusola/guides/pose_estimation && \
conda run -n texbuild tectonic -X compile pose_estimation_guide.tex
```

Where it lives in the code

Read next: `plans/pose_estimation_plan.md` (the full plan and its progress notes) and `guides/perception/perception_guide.pdf` (how predicates and the provider work).